Aula 14 – ASP.NET e Páginas Dinâmicas

Introdução ao ASP.NET

O ASP.NET é um framework desenvolvido pela Microsoft para a criação de aplicações web modernas, dinâmicas e escaláveis. Com ele, é possível desenvolver sistemas web utilizando a linguagem C# e a plataforma .NET.

No padrão ASP.NET MVC (Model-View-Controller), a aplicação é dividida em três camadas principais:

- **Model (Modelo):** Responsável pelos dados e regras de negócio.
- **View (Visão):** Responsável pela interface apresentada ao usuário.
- **Controller (Controlador):** Responsável por receber requisições, processar informações e retornar respostas.

Essa separação facilita a manutenção, organização e reutilização do código.

---

1. Controllers

O que são Controllers?

Os Controllers são responsáveis por controlar o fluxo da aplicação. Eles recebem as requisições do navegador, processam as informações necessárias e retornam uma resposta ao usuário.

Em uma aplicação ASP.NET MVC, cada Controller é uma classe que herda de `Controller`.

Exemplo de Controller

```
using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
  public IActionResult Index()
  {
     return View();
  }

  public IActionResult Sobre()
```

```
  {
      return View();
  }
}
```

Explicação

- `HomeController` é o controlador.
- `Index()` é uma Action.
- `Sobre()` é outra Action.
- `View()` retorna uma página para o usuário.

Fluxo de funcionamento

1. O usuário acessa uma URL.
2. A rota identifica qual Controller será utilizado.
3. O Controller executa a Action correspondente.
4. A Action retorna uma View.
5. A View é exibida ao usuário.

---

2. Rotas

O que são Rotas?

As rotas definem como as URLs serão associadas aos Controllers e Actions.

Sem rotas, o ASP.NET não saberia qual código executar quando uma URL fosse acessada.

Configuração de rota padrão

```
app.MapControllerRoute(
  name: "default",
  pattern: "{controller=Home}/{action=Index}/{id?}");
```

Explicação

A rota acima significa:

/controller/action/id

Exemplos:

/Home/Index
/Home/Sobre

/Produto/Listar
/Produto/Detalhes/10

Onde:

- Controller = Produto
- Action = Detalhes
- id = 10

Rotas personalizadas

Também é possível criar rotas específicas.

```
app.MapControllerRoute(
  name: "produtos",
  pattern: "catalogo/{id}",
  defaults: new { controller = "Produto", action = "Detalhes" });
```

Agora a URL:

/catalogo/15

chamará automaticamente:

ProdutoController.Detalhes(15)

Rotas por atributo

Outra forma é utilizar atributos diretamente no Controller.

```
[Route("clientes")]
public class ClienteController : Controller
{
  [Route("listar")]
  public IActionResult Listar()
  {
     return View();
  }
}
```

A URL será:

/clientes/listar

---

3. Views

O que são Views?

As Views são responsáveis pela apresentação dos dados para o usuário.

Normalmente são criadas utilizando arquivos:

.cshtml

Esses arquivos utilizam a sintaxe Razor, permitindo misturar HTML com código C#.

Exemplo de View

Arquivo:

Views/Home/Index.cshtml

Conteúdo:

<h1>Bem-vindo ao sistema</h1>

Hoje é @DateTime.Now.ToString("dd/MM/yyyy")

Saída:

Bem-vindo ao sistema

Hoje é 10/08/2025

Passando dados para a View

Utilizando ViewBag

Controller:

```
public IActionResult Index()
{
  ViewBag.Nome = "João";
  return View();
}
```

View:

```
<h2>Olá, @ViewBag.Nome</h2>
```

Resultado:

Olá, João

Utilizando ViewData

Controller:

```
public IActionResult Index()
{
  ViewData["Nome"] = "Maria";
  return View();
}
```

View:

```
<h2>@ViewData["Nome"]</h2>
```

Utilizando Razor

Exibição de variáveis

```
@{
  string nome = "Carlos";
}

<p>@nome</p>
```

Estruturas condicionais

```
@{
  int idade = 20;
}

@if (idade >= 18)
{
  <p>Maior de idade</p>
}
else
{
  <p>Menor de idade</p>
}
```

Estrutura de repetição

```
@for(int i = 1; i <= 5; i++)
{
  <p>Número @i</p>
}
```

---

4. Modelos (Models)

O que são Models?

Os Models representam os dados da aplicação e suas regras de negócio.

Eles geralmente são criados como classes C#.

Exemplo de Model

```
public class Produto
{
  public int Id { get; set; }

  public string Nome { get; set; }

  public decimal Preco { get; set; }
}
```

Utilizando o Model no Controller

```
public IActionResult Detalhes()
{
  Produto produto = new Produto
  {
    Id = 1,
    Nome = "Notebook",
    Preco = 3500
  };

  return View(produto);
}
```

Recebendo o Model na View

@model Produto

<h2>@Model.Nome</h2>

Preço: R$ @Model.Preco

Resultado:

Notebook

Preço: R$ 3500,00

Lista de Models

Controller:

```
public IActionResult Lista()
{
  List<Produto> produtos = new List<Produto>()
  {
     new Produto{Id=1, Nome="Mouse", Preco=50},
     new Produto{Id=2, Nome="Teclado", Preco=100},
     new Produto{Id=3, Nome="Monitor", Preco=800}
  };

  return View(produtos);
}
```

View:

```
@model List<Produto>

<table>
@foreach(var produto in Model)
{
  <tr>
     <td>@produto.Nome</td>
     <td>@produto.Preco</td>
  </tr>
}
</table>
```

Resumo da Aula

Controller

- Recebe as requisições.
- Processa as informações.
- Retorna Views ou dados.

Rotas

- Associam URLs aos Controllers e Actions.
- Podem ser convencionais ou por atributos.

Views

- Exibem informações ao usuário.
- Utilizam Razor (.cshtml).
- Podem receber dados do Controller.

Models

- Representam os dados do sistema.
- Contêm propriedades e regras de negócio.
- São enviados para as Views pelos Controllers.

Exercícios

1. O que é um Controller no ASP.NET MVC e qual sua principal função?

---

2. Qual a diferença entre Model, View e Controller?

---

3. Analise a rota abaixo e explique seu funcionamento:

pattern: "{controller=Home}/{action=Index}/{id?}"

---

4. Crie um Controller chamado `ProdutoController` contendo uma Action chamada `Listar`.

---

5. Crie uma rota personalizada para acessar:

/produtos/10

direcionando para a Action `Detalhes` do `ProdutoController`.

---

6. Escreva uma View que exiba a mensagem:

Bem-vindo ao ASP.NET MVC

---

7. Utilize `ViewBag` para enviar o nome "Pedro" do Controller para a View e exiba-o na tela.

---

8. Crie uma classe Model chamada `Cliente` contendo:

- Id
- Nome
- Email

---

9. Crie uma Action que envie um objeto `Cliente` para uma View.

---

10. Crie uma View fortemente tipada que receba um objeto `Cliente` e exiba todas as suas propriedades.